

Métodos de Ordenação: Merge e Quick (Sort)

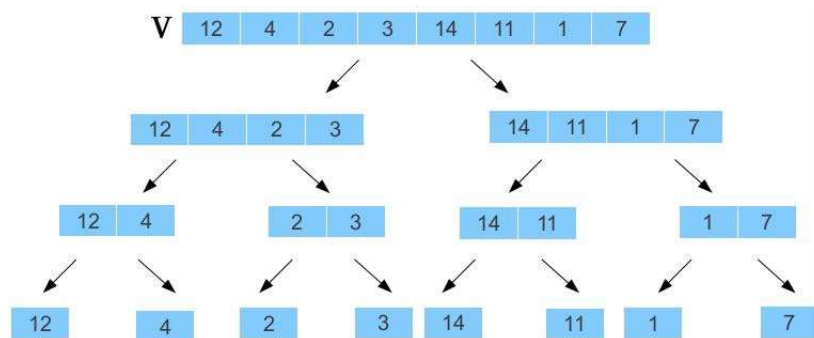
MergeSort (intercalação direta) - Ideia

- divida o vetor em 2 subvetores (ao meio) recursivamente até ele ter o tamanho 1.
- Intercale pares de elementos adjacentes. Repita esse processo até restar apenas um arquivo de tamanho n .

MergeSort (intercalação direta) - Ideia

- Dividir e Conquistar;
- Divide, recursivamente, o conjunto de dados até que o subconjunto possua **1** elemento
- Combina **2** subconjuntos de forma a obter **1** conjunto maior e ordenado
- Esse processo se repete até que exista apenas **1** conjunto.

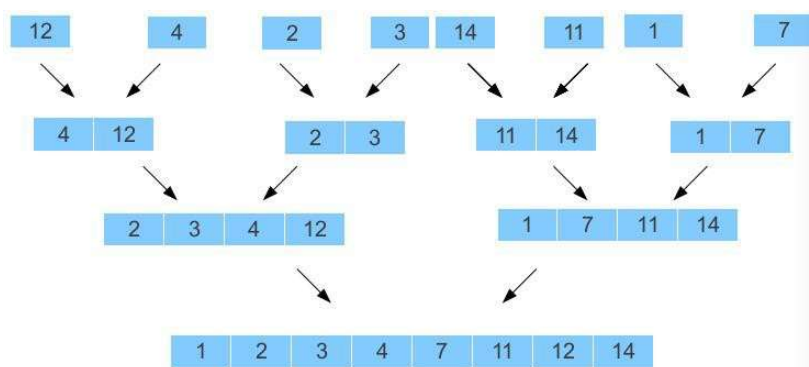
MergeSort (intercalação direta) - Exemplo



MergeSort (intercalação direta) - Ideia

- divida o vetor em 2 subvetores (ao meio) recursivamente até ele ter o tamanho 1.
- **Intercale pares de elementos adjacentes.** Repita esse processo até restar apenas um arquivo de tamanho n .

MergeSort (intercalação direta) - Exemplo



Intercultural Computer Science Education
https://www.youtube.com/watch?v=XaqR3G_NVoo

MergeSort (intercalação direta) Python

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    meio = len(arr) // 2
    esquerda = merge_sort(arr[:meio])
    direita = merge_sort(arr[meio:])
    return merge(esquerda, direita)

def merge(esquerda, direita):
    resultado = []
    i = j = 0
    while i < len(esquerda) and j < len(direita):
        if esquerda[i] < direita[j]:
            resultado.append(esquerda[i])
            i += 1
        else:
```

MergeSort (intercalação direta) Python

```
            resultado.append(direita[j])
            j += 1

    resultado.extend(esquerda[i:])
    resultado.extend(direita[j:])

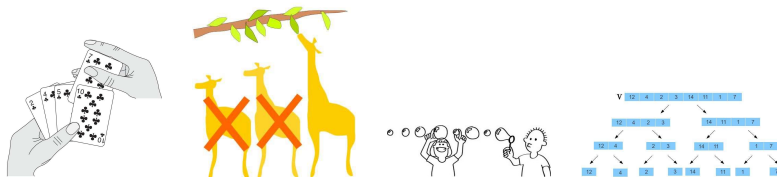
    return resultado

if __name__ == "__main__":
    lista = [38, 27, 43, 3, 9, 82, 10]
    ordenada = merge_sort(lista)
    print("Lista original:", lista)
    print("Lista ordenada:", ordenada)
```

Ordenação

Vimos os métodos de ordenação:

- InsertionSort (n^2)
- SelectionSort (n^2)
- BubbleSort (n^2)
- MergeSort ($n * \log_2 n$)



Ordenação

Veremos agora, o método de ordenação:

- QuickSort (pior caso n^2)

Ordenação

- A maioria dos métodos de ordenação é baseado em **comparação de chaves**;
 - Exemplos: insertionsort, selectionsort, BubbleSort, MergeSort, **QuickSort**;
- Existem métodos de ordenação que utilizam o princípio da **distribuição**;
 - Exemplos: radixsort, bucketsort, etc;

Ordenação rápida (QuickSort)

A idéia do QuickSort é:

- **Dividir:** O arranjo $A[p..r]$ é particionado em dois subarranjos $A[p..q - 1]$ e $A[q + 1..r]$ tais que cada elemento de $A[p..q - 1]$ é menor ou igual a $A[q]$ que, por sua vez, é menor ou igual a cada elemento de $A[q + 1..r]$. O índice q é calculado como parte do procedimento;

Ordenação rápida (QuickSort)

A idéia do QuickSort é:

- **Dividir:** O arranjo $A[p..r]$ é particionado em dois subarranjos $A[p..q - 1]$ e $A[q + 1..r]$ tais que cada elemento de $A[p..q - 1]$ é menor ou igual a $A[q]$ que, por sua vez, é menor ou igual a cada elemento de $A[q + 1..r]$. O índice q é calculado como parte do procedimento;
- **Conquistar:** Os 2 subarranjos $A[p..q - 1]$ e $A[q + 1..r]$ são ordenados por chamadas recursivas;

Ordenação rápida (QuickSort)

A idéia do QuickSort é:

- **Dividir:** O arranjo $A[p..r]$ é particionado em dois subarranjos $A[p..q - 1]$ e $A[q + 1..r]$ tais que cada elemento de $A[p..q - 1]$ é menor ou igual a $A[q]$ que, por sua vez, é menor ou igual a cada elemento de $A[q + 1..r]$. O índice q é calculado como parte do procedimento;
- **Conquistar:** Os 2 subarranjos $A[p..q - 1]$ e $A[q + 1..r]$ são ordenados por chamadas recursivas;
- **Combinar:** Os subarranjos são ordenados localmente, então não é necessário combinar;

Ordenação rápida (QuickSort) - Exemplo

Legenda: pivô: ; 1ª partição: ; 2ª partição: 

2	8	7	1	3	5	6	4
---	---	---	---	---	---	---	---

Ordenação rápida (QuickSort) - Exemplo

Legenda: pivô: ; 1ª partição: ; 2ª partição: 

2	8	7	1	3	5	6	4
2	8	7	1	3	5	6	

Ordenação rápida (QuickSort) - Exemplo

Legenda: pivô: pivô; 1ª partição: ; 2ª partição:

2	8	7	1	3	5	6	4
2	8	7	1	3	5	6	4
2	8	7	1	3	5	6	4

Ordenação rápida (QuickSort) - Exemplo

Legenda: pivô: pivô; 1ª partição: ; 2ª partição:

2	8	7	1	3	5	6	4
2	8	7	1	3	5	6	4
2	8	7	1	3	5	6	4
2	8	7	1	3	5	6	4

Ordenação rápida (QuickSort) - Exemplo

Legenda: **pivô**; 1ª partição: ; 2ª partição: 

2	8	7	1	3	5	6	4
2	8	7	1	3	5	6	4
2	8	7	1	3	5	6	4
2	8	7	1	3	5	6	4
2	8	7	1	3	5	6	4

Ordenação rápida (QuickSort) - Exemplo

Legenda: **pivô**; 1ª partição: ; 2ª partição: 

2	8	7	1	3	5	6	4
2	8	7	1	3	5	6	4
2	8	7	1	3	5	6	4
2	8	7	1	3	5	6	4
2	8	7	1	3	5	6	4
2	1	7	8	3	5	6	4

Ordenação rápida (QuickSort) - Exemplo

Legenda: **pivô**; 1ª partição: ; 2ª partição:

2	8	7	1	3	5	6	4
2	8	7	1	3	5	6	4
2	8	7	1	3	5	6	4
2	8	7	1	3	5	6	4
2	8	7	1	3	5	6	4
2	1	7	8	3	5	6	4
2	1	3	8	7	5	6	4

Ordenação rápida (QuickSort) - Exemplo

Legenda: **pivô**; 1ª partição: ; 2ª partição:

2	8	7	1	3	5	6	4
2	8	7	1	3	5	6	4
2	8	7	1	3	5	6	4
2	8	7	1	3	5	6	4
2	8	7	1	3	5	6	4
2	1	7	8	3	5	6	4
2	1	3	8	7	5	6	4
2	1	3	8	7	5	6	4

Ordenação rápida (QuickSort) - Exemplo

Legenda: **pivô**; 1ª partição: ; 2ª partição: 

2	8	7	1	3	5	6	4
2	8	7	1	3	5	6	4
2	8	7	1	3	5	6	4
2	8	7	1	3	5	6	4
2	8	7	1	3	5	6	4
2	1	7	8	3	5	6	4
2	1	3	8	7	5	6	4
2	1	3	8	7	5	6	4
2	1	3	8	7	5	6	4

Ordenação rápida (QuickSort) - Exemplo

Legenda: **pivô**; 1ª partição: ; 2ª partição: 

2	8	7	1	3	5	6	4
2	8	7	1	3	5	6	4
2	8	7	1	3	5	6	4
2	8	7	1	3	5	6	4
2	8	7	1	3	5	6	4
2	1	7	8	3	5	6	4
2	1	3	8	7	5	6	4
2	1	3	8	7	5	6	4
2	1	3	8	7	5	6	4
2	1	3	4	7	5	6	8

Ordenação rápida (QuickSort) - Exemplo

Legenda: **pivô**; 1ª partição: ; 2ª partição: 

2	1	3	4	7	5	6	8
---	---	---	---	---	---	---	---

- Todos elementos da 1ª partição são menores ou iguais ao **pivô**
- Todos elementos da 2ª partição são maiores que o **pivô**

Ordenação rápida (QuickSort) - Exemplo

Legenda: **pivô**; 1ª partição: ; 2ª partição: 

2	1	3	4	7	5	6	8
---	---	---	---	---	---	---	---

- Todos elementos da 1ª partição são menores ou iguais ao **pivô**
- Todos elementos da 2ª partição são maiores que o **pivô**

Intercultural Computer Science Education
<https://www.youtube.com/watch?v=ywWBy6J5gz8>

Ordenação rápida (QuickSort) - Python

```
def quicksort(arr):  
    if len(arr) <= 1:  
        return arr  
  
    pivot = arr[len(arr) // 2]  
  
    menores = [x for x in arr if x < pivot]  
    iguais = [x for x in arr if x == pivot]  
    maiores = [x for x in arr if x > pivot]
```

Ordenação rápida (QuickSort) - Python

```
    return quicksort(menores) + iguais +  
    quicksort(maiores)  
  
if __name__ == "__main__":  
    lista = [10, 7, 8, 9, 1, 5]  
    print("Original:", lista)  
    print("Ordenada:", quicksort(lista))
```